Requirements and Analysis

- Definition
  - Mozaic, the system, and Mozaic system refers software system that this project is going to develops, launches, and operates.
- Goal
  - This document serves as requirements specification of the system
  - This document serves as the technical terminology of the project.

1. Major concepts and requirements:

- A user should be represented by a wallet, technically. If a user has multiple wallets, the user should be represented by the wallet that is currently involved with the system by the user's current use case.
- A user can **deposit** their assets in any listed token format on any listed chain from their wallet to the system wallet.
- The system should **stake** the deposited assets from system wallet(s) to staking pools.
- If the system **stake**s rewards generated by staking, the system **compound**s rewards.
- *The system should not be able to track which part of user's deposit is staked on which staking pool and performs how well.*
- *The system should be able convert the deposits to any listed token on any listed chain, to maximize profit.*
- **Staking stock** is the sum of the total currently staked assets and total currently pending rewards on all staking pools.
- When the system **stake**s asset that a user **deposit**ed, the system returns LP tokens to the user. The amount of the returned LP token should represent the newly staked asset in the **Staking stock** immediately after the asset is staked.
- A user can **withdraw** assets from the **Staking stock**, in any listed token format on any listed chain, by first returning LP tokens from their wallet to the system wallet.
- The amount of asset that is **withdraw**n is the portion of **Staking stock** that is represented by the returned LP tokens immediately before the asset is **withdraw**n.
- The LP token may exist on a single listed chain, some listed chains, or all listed chains. If the LP token exists on all listed chains, it is an omnichain LP.
- All LP token versions should always have the save value.

- A user can transfer one version of LP token from their wallet to any version of LP token on any wallet or account.
- The system should make sure that only the system can **stake** deposited asset, subtract from **stake**d asset, collect reward, and **compound** reward.

2. Use cases

From the major concepts and requirements, we can deduce the following use cases.

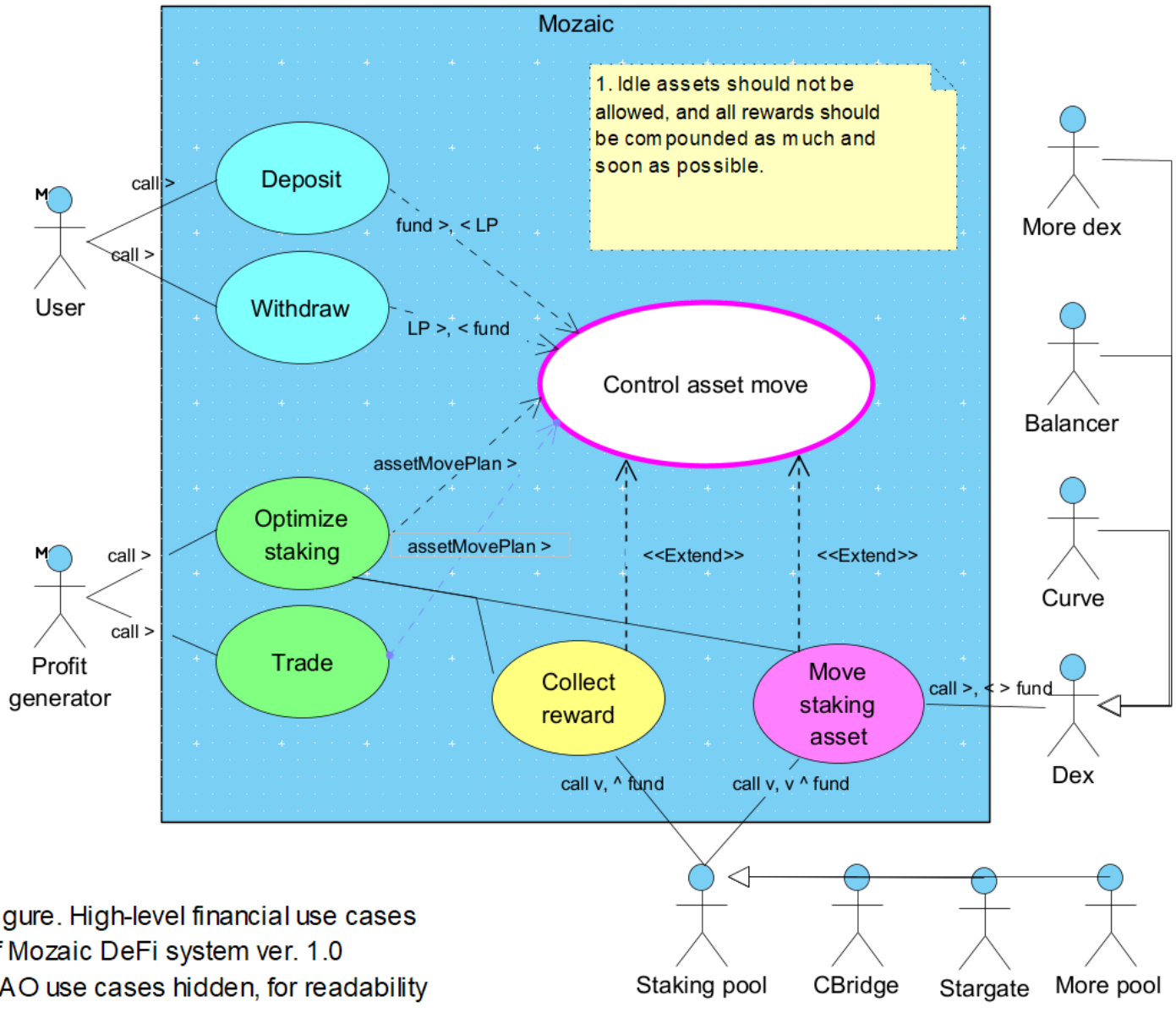

Figure. High-level financial use cases of Mozaic DeFi system ver. 1.0
DAO use cases hidden, for readability

The use cases and external actors are described as below:

- **Compound**: This hidden use case compounds rewards. All rewards should be compounded as much and soon as possible.
- **Control asset move**: This use case checks, carries out, and keeps track of asset moves. The assets managed by the system can only be moved by this use case transparently.
- **Deposit**: This use case deposits the user's assets in the system. **User** invokes this use case in the hope that the system will **Optimize staking** of the deposited assets for them. **User** should be able to deposit any listed token on any listed chain. This use case includes **Control asset move**.
- **Withdraw**: This use case withdraws from **Staking stock**. **User** should be able to withdraw any listed token on any listed chain, no matter what token on which chain they deposited. (The deposited assets are increased by perpetual compounding of rewards.) **User** invokes this use case. It includes **Control asset move**.
- **Optimize staking**: This use case upgrades the staking portfolio to maximize rewards. **Profit generator**, a role of the system, invokes this use case *either on a regular basis or at randomly picked times*. This use case includes **Control asset move** and **Compound**. *By providing **Control asset move** with **assetMovePlan**, this use case effectively prevents it from being involved with finding optimal staking portfolio.*
- **Trade**: This use case swaps idle assets to get profit from price changes. It calls **Dex**. *By providing **Control asset move** with **assetMovePlan**, this use case effectively prevents it from being involved with finding optimal trading orders.*
- **Collect reward**: This use case collects rewards from Staking pools. Use case **Control asset move**, when it is working included **Optimize staking**, is extended by this use case. This use case calls **Staking pool**.
- **Move staking asset**: This use case moves assets to/between/from, **Staking pools**. **Control asset move**, when it is working included **Optimize staking**, is extended by this use case. This use case calls **Staking pool**.
- **Dex**: This actor is a smart contract that swaps between assets. Examples are pairs on Curve and Balancer DeFis.
- **Staking pool**: This actor is a smart contract that allocates reward to assets that are staked in it. Examples are farming pools on CBridge and Stargate DeFis.

3. Lawful responsibility and freedom of the system

- **Guaranteed Withdrawal**: The system is legally obliged to allow a user to withdraw the full amount of assets that the user have deposited in the system, in XXX hours after the user's request.

- **Unguaranteed Profit**: The system is not legally obliged to return profit to a user who deposited assets in the system.
- **Guaranteed Asset Control**: No person or entity other than the system can stake deposited asset, subtract from staked asset, collect reward, and compound reward.
- **Transparent Logging**: All asset/profit creation/move must be correctly logged in/at an available format and place.
- **Transparent Profit**: The division of profit among users must be in accordance with the promise transparently.